

Optimizing Common Table Expressions in Apache Hive with Calcite

Stamatis Zampetakis



About me

Stamatis Zampetakis

Senior Staff Software Engineer @ Cloudera, Query compiler team

PMC member of Apache Calcite & Apache Hive

PhD in Data Management, INRIA & Paris-Sud University



Common Table Expressions

Classify and return departments (the names) based on the average employee salary (HIGH > 100K, LOW < 100K)

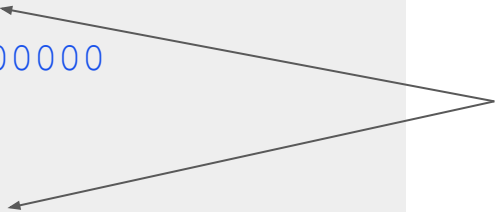
```
SELECT d.name, 'HIGH'
FROM emps e
INNER JOIN depts d ON e.deptno = d.deptno
GROUP BY d.name
HAVING AVG(e.salary) >= 100000
UNION
SELECT d.name, 'LOW'
FROM emps e
INNER JOIN depts d ON e.deptno = d.deptno
GROUP BY d.name
HAVING AVG(e.salary) < 100000
```

Common Table Expressions

Classify and return departments (the names) based on the average employee salary (HIGH > 100K, LOW < 100K)

```
SELECT d.name, 'HIGH'
FROM emps e
INNER JOIN depts d ON e.deptno = d.deptno
GROUP BY d.name
HAVING AVG(e.salary) >= 100000
UNION
SELECT d.name, 'LOW'
FROM emps e
INNER JOIN depts d ON e.deptno = d.deptno
GROUP BY d.name
HAVING AVG(e.salary) < 100000
```

Common Table Expression
(CTE)



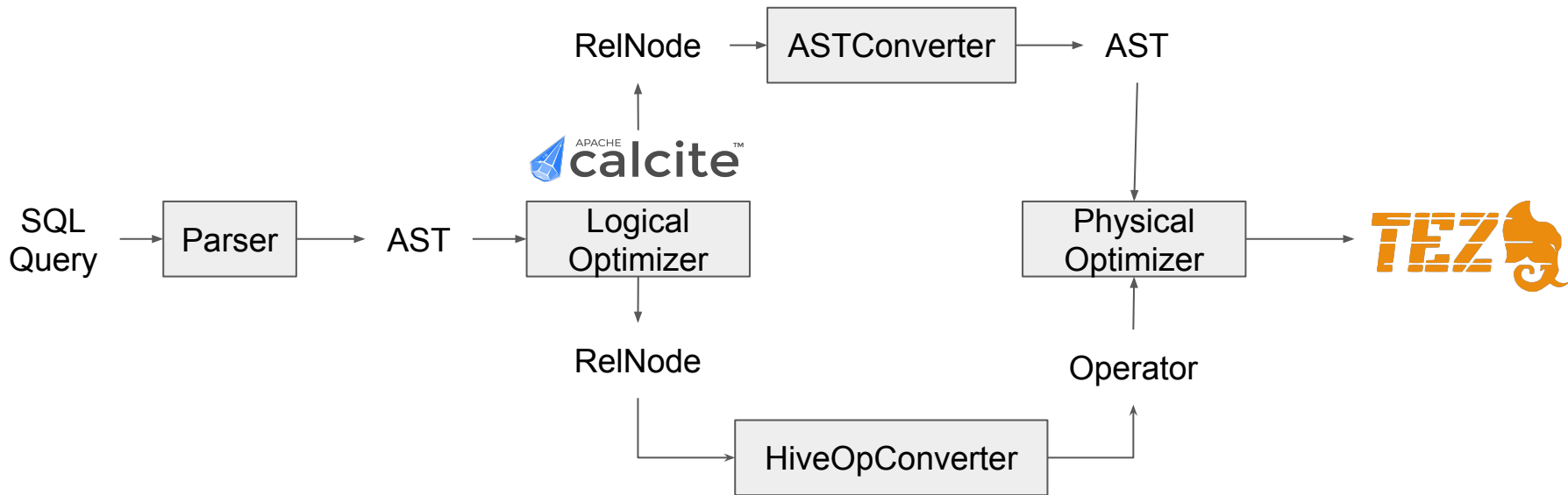
Common Table Expressions

- CTEs appear very frequently in SQL queries
 - **Explicitly** defined by users to facilitate query formulations
 - **Implicitly** generated by business intelligence (BI) and decision support tools
- Computation reuse & caching are crucial for cost* reduction
- Cost increase with data increase

*cost may refer to resource utilization, response time, monetary value, etc.

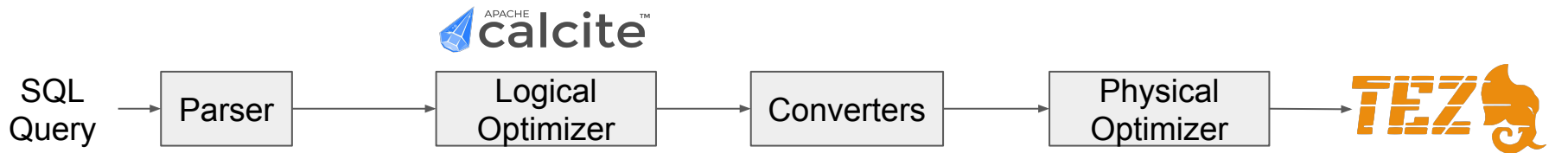


Hive Planner Overview





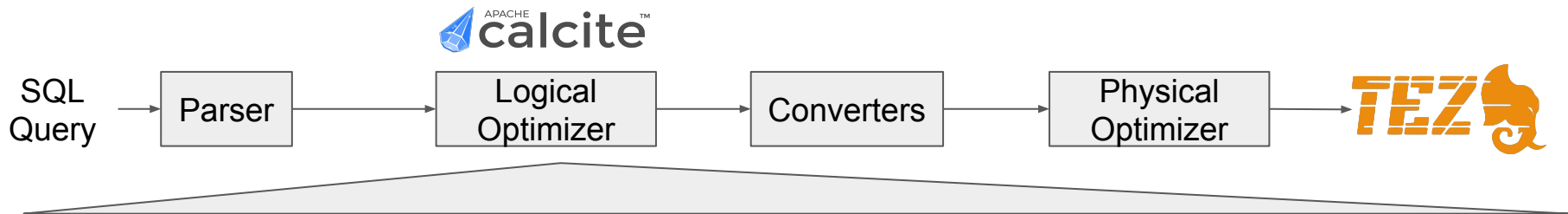
Hive Planner Overview



1. Subquery removal
2. Query decorrelation
3. Pre-join ordering
4. MV rewriting
5. Join ordering
6. Post-join ordering

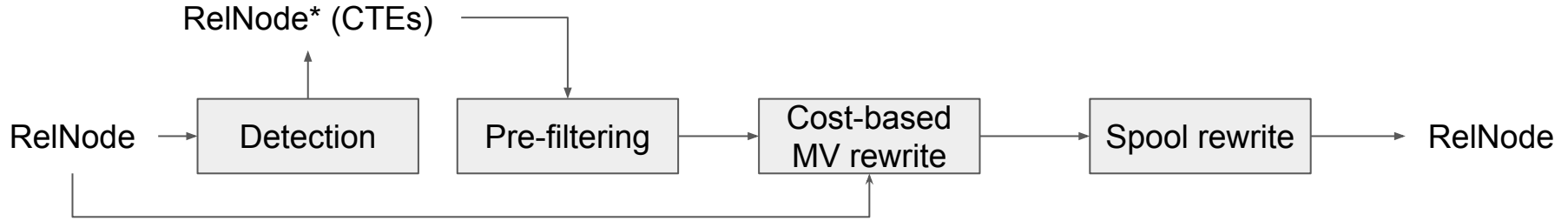


Hive Planner Overview

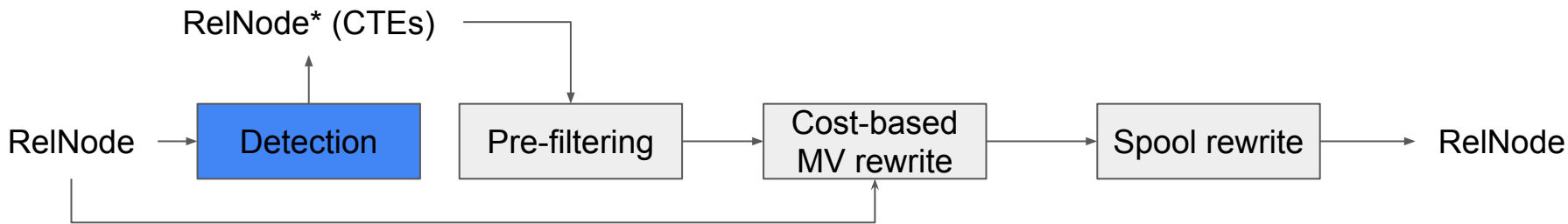


1. Subquery removal
2. Query decorrelation
3. Pre-join ordering
4. MV rewriting
5. Join ordering
6. Post-join ordering
7. **CTE rewriting [NEW]**

CTE Rewriting phase



CTE Rewriting phase - Detection



Detection logic must implement the [CommonTableExpressionSuggester](#) interface

Pluggable via [hive.optimize.cte.suggester.class](#) property

[CommonTableExpressionIdentitySuggester](#) is a baseline implementation available in Hive

- Detects identical subtrees / No transformations
- Uses [CommonRelSubExprRule](#) class (HepPlanner only)

CTE Rewriting phase - Detection

```
SELECT d.name, 'HIGH'
FROM emps e
INNER JOIN depts d ON e.deptno = d.deptno
GROUP BY d.name
HAVING AVG(e.salary) >= 100000
UNION
SELECT d.name, 'LOW'
FROM emps e
INNER JOIN depts d ON e.deptno = d.deptno
GROUP BY d.name
HAVING AVG(e.salary) < 100000
```

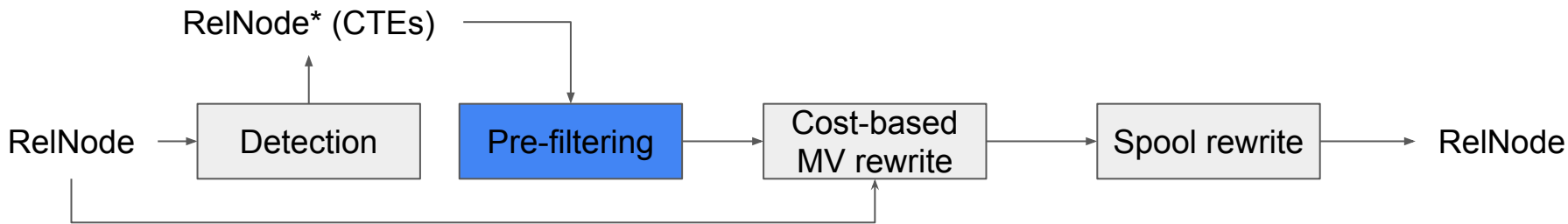
CTE Rewriting phase - Detection

```
HiveAggregate(group=[{0, 1}])
  HiveProject(name=[$0], _o__c1=[$1])
    HiveUnion(all=[true])
      HiveProject(name=[$0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
        HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
          HiveAggregate(group=[{3}], agg#0=[sum($1)], agg#1=[count($1)])
            HiveJoin(condition=[=($0, $2)], joinType=[inner], algorithm=[none], cost=[not available])
              HiveProject(deptno=[$1], salary=[$3])
                HiveFilter(condition=[IS NOT NULL($1)])
                  HiveTableScan(table=[[default, emps]], table:alias=[e])
                    HiveProject(deptno=[$0], name=[$1])
                      HiveFilter(condition=[IS NOT NULL($0)])
                        HiveTableScan(table=[[default, depts]], table:alias=[d])
HiveProject(name=[$0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
  HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
    HiveAggregate(group=[{3}], agg#0=[sum($1)], agg#1=[count($1)])
      HiveJoin(condition=[=($0, $2)], joinType=[inner], algorithm=[none], cost=[not available])
        HiveProject(deptno=[$1], salary=[$3])
          HiveFilter(condition=[IS NOT NULL($1)])
            HiveTableScan(table=[[default, emps]], table:alias=[e])
              HiveProject(deptno=[$0], name=[$1])
                HiveFilter(condition=[IS NOT NULL($0)])
                  HiveTableScan(table=[[default, depts]], table:alias=[d])
```

CTE Rewriting phase - Detection

```
HiveAggregate(group=[{0, 1}])
HiveProject(name=[$0], _o__c1=[$1])
HiveUnion(all=[true])
HiveProject(name=[$0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
HiveAggregate(group=[{3}], agg#0=[sum($1)], agg#1=[count($1)])
HiveJoin(condition=[=($0, $2)], joinType=[inner], algorithm=[none], cost=[not available])
HiveProject(deptno=[$1], salary=[$3])
HiveFilter(condition=[IS NOT NULL($1)])
HiveTableScan(table=[[default, emps]], table:alias=[e])
HiveProject(deptno=[$0], name=[$1])
HiveFilter(condition=[IS NOT NULL($0)])
HiveTableScan(table=[[default, depts]], table:alias=[d])
HiveProject(name=[$0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
HiveAggregate(group=[{3}], agg#0=[sum($1)], agg#1=[count($1)])
HiveJoin(condition=[=($0, $2)], joinType=[inner], algorithm=[none], cost=[not available])
HiveProject(deptno=[$1], salary=[$3])
HiveFilter(condition=[IS NOT NULL($1)])
HiveTableScan(table=[[default, emps]], table:alias=[e])
HiveProject(deptno=[$0], name=[$1])
HiveFilter(condition=[IS NOT NULL($0)])
HiveTableScan(table=[[default, depts]], table:alias=[d])
```

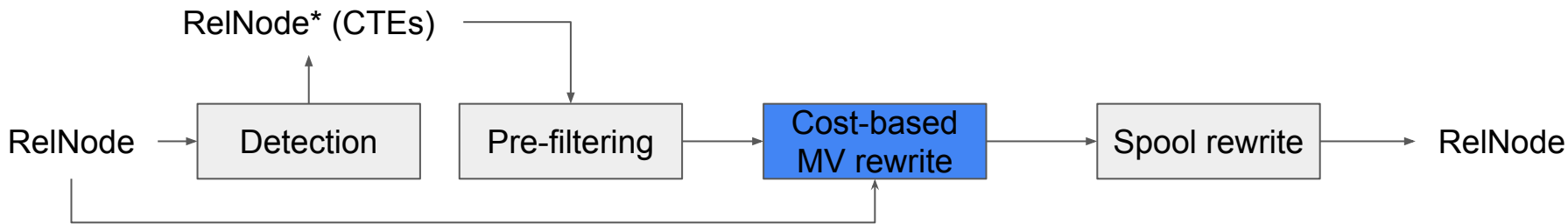
CTE Rewriting phase - Pre-filtering



Apply Hive specific checks to CTEs

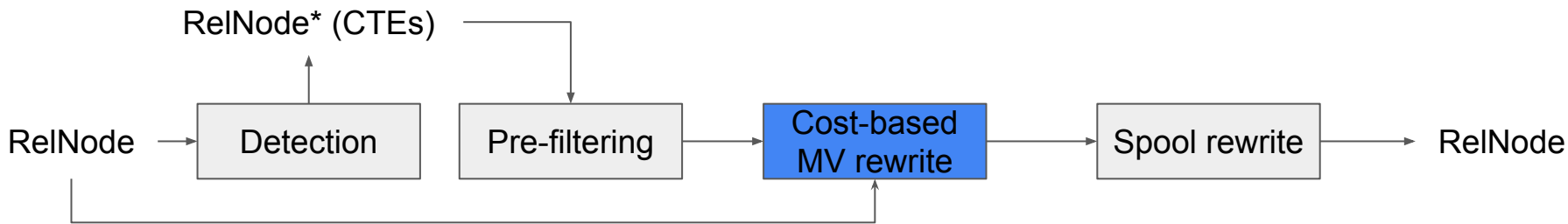
1. Retain only full aggregates if [hive.optimize.cte.materialize.full.aggregate.only](#) enabled
2. Remove untyped nulls; not supported ([HIVE-11217](#))

CTE Rewriting phase - MV Rewrite



1. Wrap **CTEs** (RelNode) into **materialized views** (RelOptMaterialization)
2. Rewrite original plan using Hive's MV rewriting logic
 - a. Apply pre-rewriting transformations: HepPlanner + Hive specific rules
 - b. Perform cost-based MV rewriting using:
 - i. HiveVolcanoPlanner (overrides cost model for views)
 - ii. Extensions of **MaterializedViewRule** for Hive
 - iii. Hive specific rules

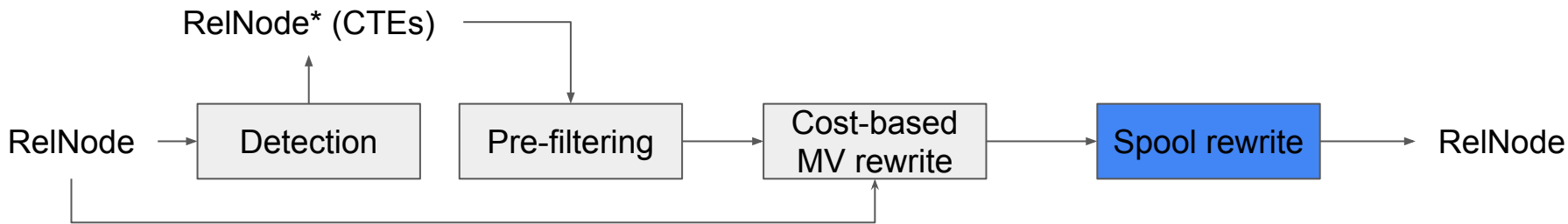
CTE Rewriting phase - MV Rewrite



1. Wrap **CTEs** (RelNode) into **materialized views** (RelOptMaterialization)
2. Rewrite original plan using Hive's MV rewriting logic

```
HiveAggregate(group=[{0, 1}])
  HiveUnion(all=[true])
    HiveProject(name=[$0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
    HiveProject(name=[$0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
```


CTE Rewriting phase - Spool Rewrite



1. Introduce **Spool** operator marking the CTE **producer**
2. Remove CTE references under the **hive.optimize.cte.materialize.threshold**

CTE Rewriting phase - Spool Rewrite

1. Introduce **Spool** operator marking the CTE **producer**
2. Remove CTE references under the **hive.optimize.cte.materialize.threshold**

```
HiveAggregate(group=[{0, 1}])
  HiveUnion(all=[true])
    HiveProject(name=[0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
    HiveProject(name=[0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
```

CTE Rewriting phase - Spool Rewrite

1. Introduce **Spool** operator marking the CTE **producer**
2. Remove CTE references under the **hive.optimize.cte.materialize.threshold**

```
HiveAggregate(group=[{0, 1}])
  HiveUnion(all=[true])
    HiveProject(name=[0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
    HiveProject(name=[0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
```

CTE Rewriting phase - Spool Rewrite

1. Introduce **Spool** operator marking the CTE **producer**
2. Remove CTE references under the **hive.optimize.cte.materialize.threshold**

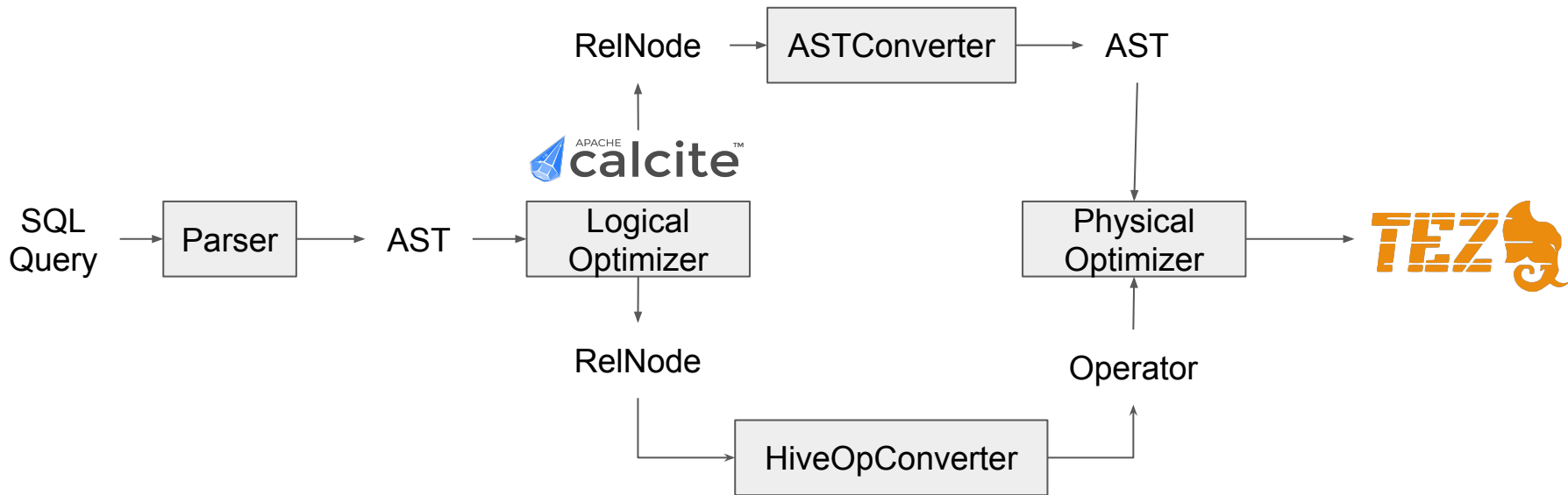
```
HiveAggregate(group=[{0, 1}])
  HiveUnion(all=[true])
    HiveProject(name=[0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableSpool(table=[[cte, cte_suggestion_0]])
          HiveProject(name=[0], $f1=[1], $f2=[2])
            HiveAggregate(group=[{3}], agg#0=[sum($1)], agg#1=[count($1)])
              HiveJoin(condition=[=(0, 2)], joinType=[inner], algorithm=[none])
                HiveProject(deptno=[1], salary=[3])
                  HiveFilter(condition=[IS NOT NULL($1)])
                    HiveTableScan(table=[[default, emps]], table:alias=[e])
                      HiveProject(deptno=[0], name=[1])
                        HiveFilter(condition=[IS NOT NULL($0)])
                          HiveTableScan(table=[[default, depts]], table:alias=[d])
HiveProject(name=[0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
  HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
    HiveTableScan(table=[[cte, cte_suggestion_0]], table:alias=[cte_suggestion_0])
```

CTE Rewriting phase - Spool Rewrite

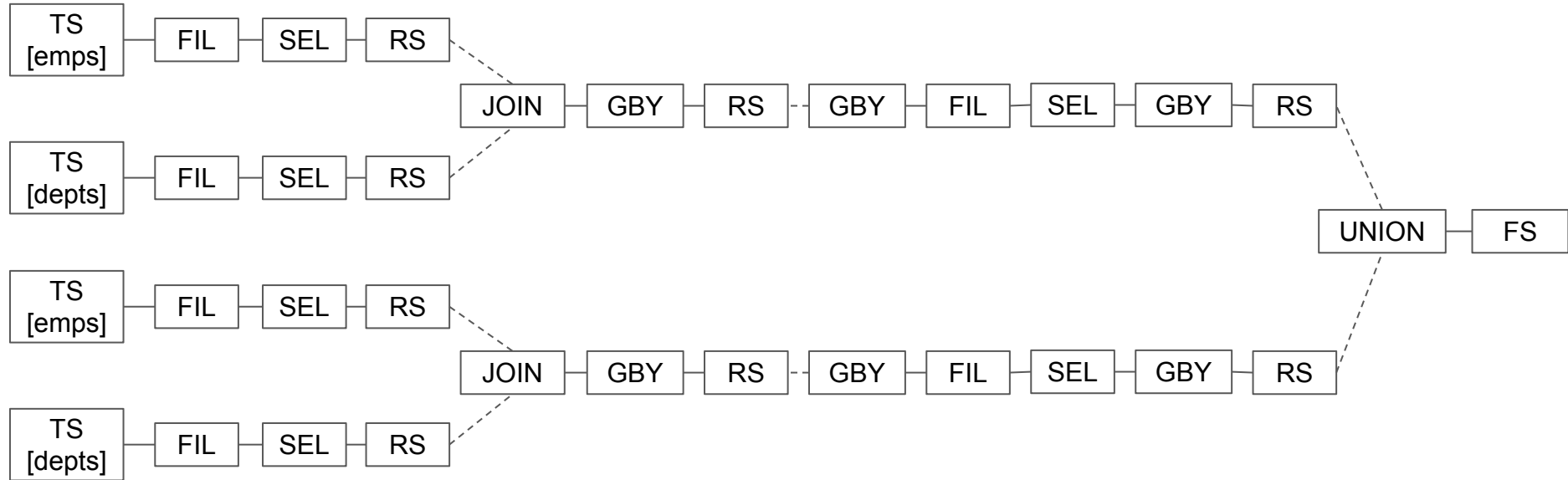
1. Introduce **Spool** operator marking the CTE **producer**
2. Remove CTE references under the **hive.optimize.cte.materialize.threshold**

```
HiveAggregate(group=[{0, 1}])
  HiveUnion(all=[true])
    HiveProject(name=[0], _o__c1=[_UTF-16LE'HIGH':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
      HiveFilter(condition=[>=(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
        HiveTableSpool(table=[cte, cte_suggestion_0])
          HiveProject(name=[0], $f1=[1], $f2=[2])
            HiveAggregate(group=[{3}], agg#0=[sum($1)], agg#1=[count($1)])
              HiveJoin(condition=[=($0, $2)], joinType=[inner], algorithm=[none])
                HiveProject(deptno=[1], salary=[3])
                  HiveFilter(condition=[IS NOT NULL($1)])
                    HiveTableScan(table=[[default, emps]], table:alias=[e])
                  HiveProject(deptno=[0], name=[1])
                    HiveFilter(condition=[IS NOT NULL($0)])
                      HiveTableScan(table=[[default, depts]], table:alias=[d])
                HiveProject(name=[0], _o__c1=[_UTF-16LE'LOW':VARCHAR(2147483647) CHARACTER SET "UTF-16LE"])
                  HiveFilter(condition=[<(CAST(/($1, $2)):DECIMAL(12, 6), 100000:DECIMAL(6, 0))])
                    HiveTableScan(table=[cte, cte_suggestion_0], table:alias=[cte_suggestion_0])
```

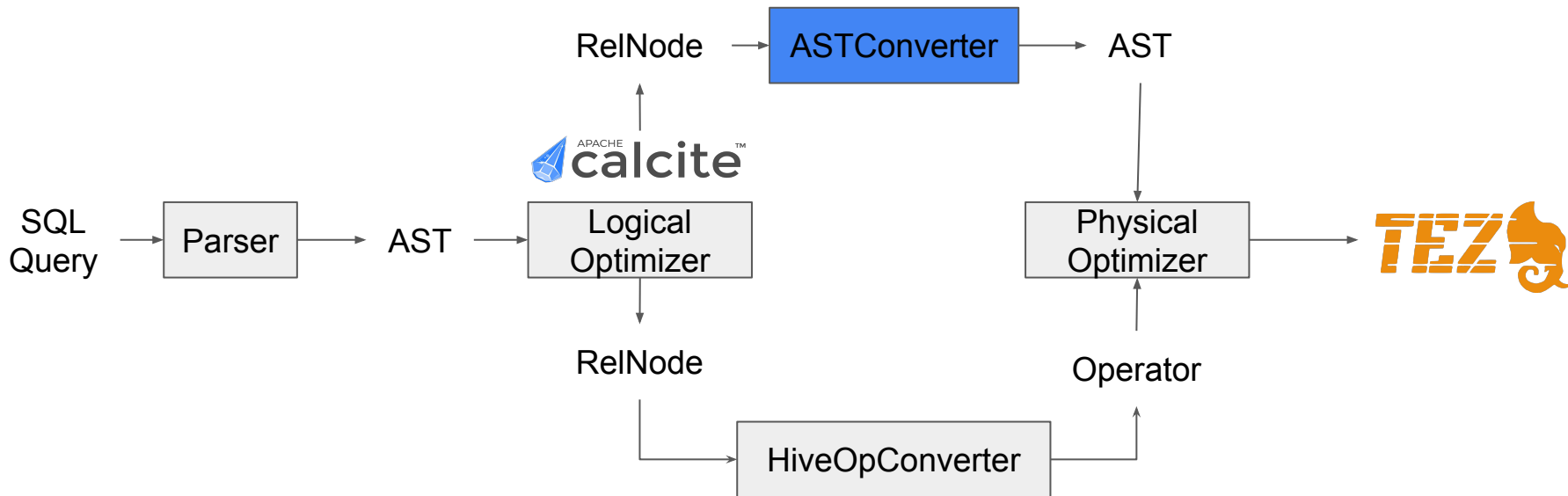
Spool to Physical



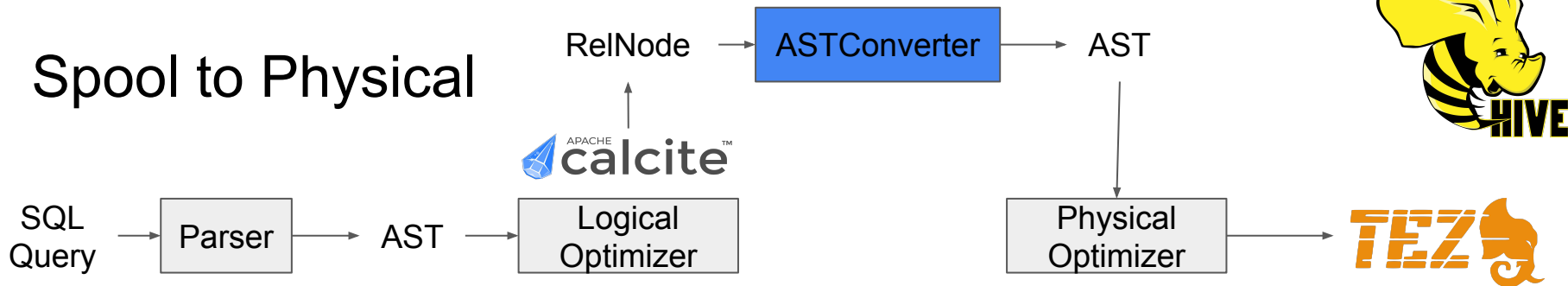
Spool to Physical



Spool to Physical



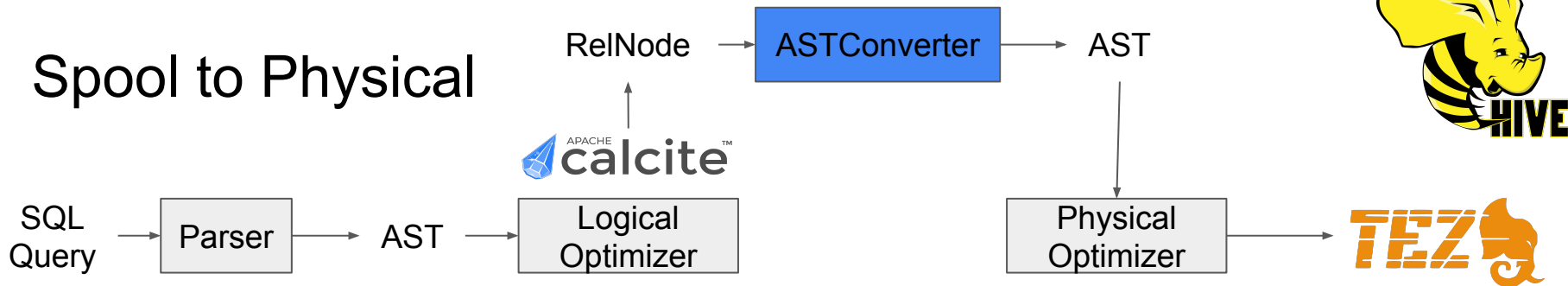
Spool to Physical



```
HiveTableSpool(table=[[cte, cte_suggestion_0]])
HiveProject(name=[0], $f1=[1], $f2=[2])
  HiveAggregate(group=[{3}], agg#0=[sum(1)], agg#1=[count(1)])
    HiveJoin(condition=[(0, 2)], joinType=[inner], algorithm=[none])
      HiveProject(deptno=[1], salary=[3])
        HiveFilter(condition=[IS NOT NULL(1)])
          HiveTableScan(table=[[default, emps]], table:alias=[e])
        HiveProject(deptno=[0], name=[1])
          HiveFilter(condition=[IS NOT NULL(0)])
            HiveTableScan(table=[[default, depts]], table:alias=[d])
```

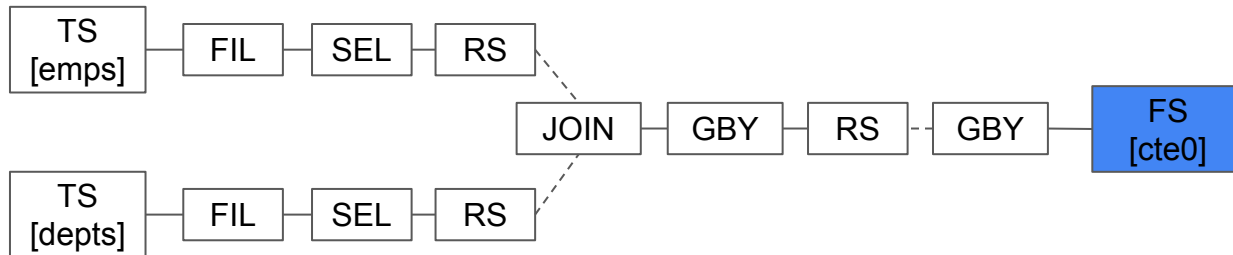
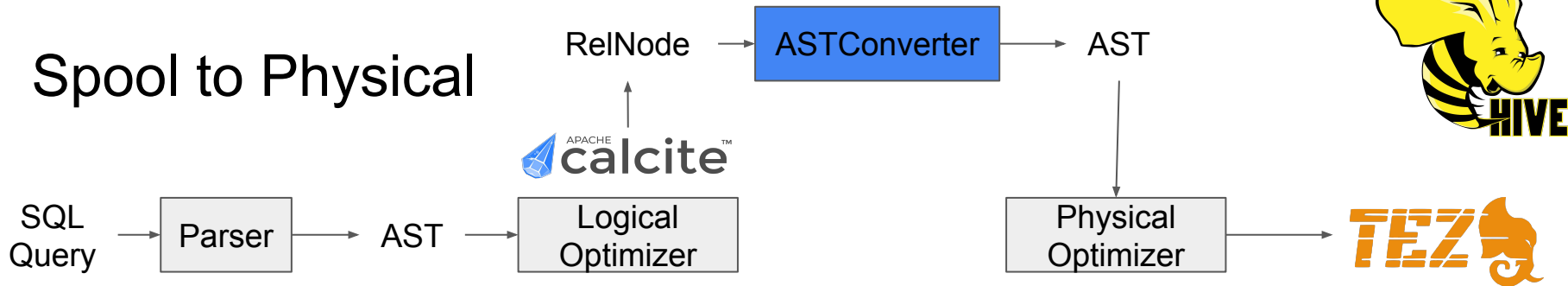
```
WITH cte_suggestion_0 AS (
  SELECT d.name as d_name, SUM(e.salary) as sum_sal, COUNT(e.salary) as cnt_sal
  FROM emps e
  INNER JOIN depts d ON e.deptno = d.deptno
  WHERE e.deptno IS NOT NULL AND d.deptno IS NOT NULL
  GROUP BY d.name
)
```

Spool to Physical

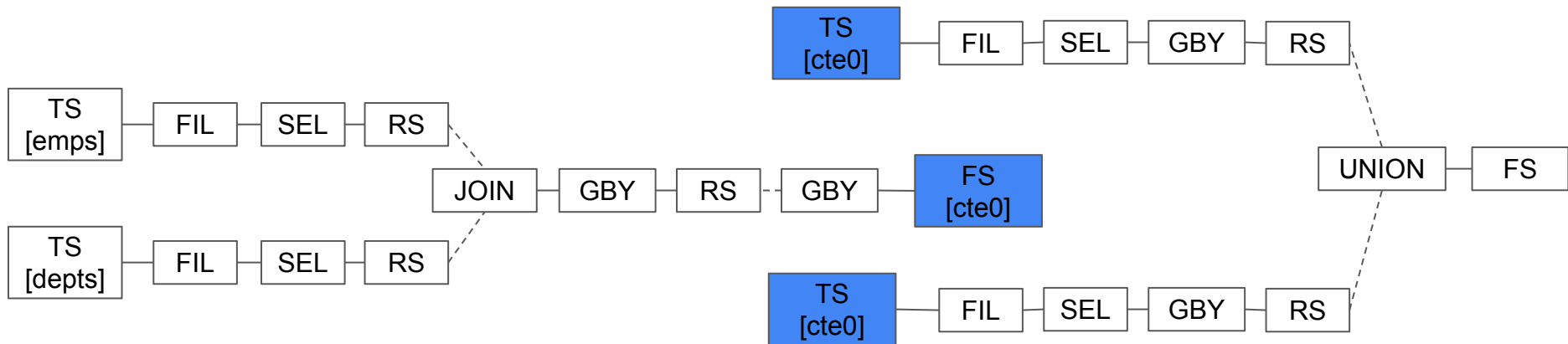
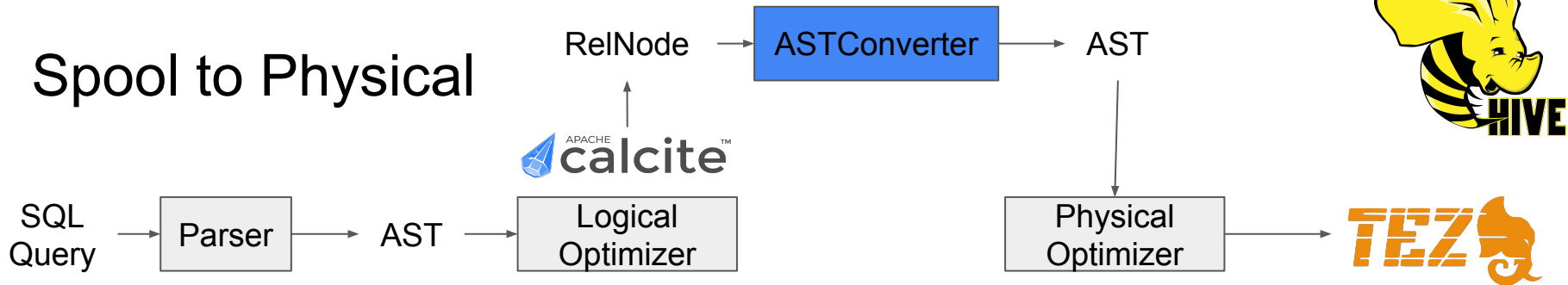


```
WITH cte_suggestion_0 AS (  
    SELECT d.name as d_name, SUM(e.salary) as sum_sal, COUNT(e.salary) as cnt_sal  
    FROM emps e  
    INNER JOIN depts d ON e.deptno = d.deptno  
    WHERE e.deptno IS NOT NULL AND d.deptno IS NOT NULL  
    GROUP BY d.name  
)  
SELECT d_name, 'HIGH' FROM cte_suggestion_0 WHERE sum_sal/cnt_sal >= 100000  
UNION  
SELECT d_name, 'LOW' FROM cte_suggestion_0 WHERE sum_sal/cnt_sal < 100000;
```

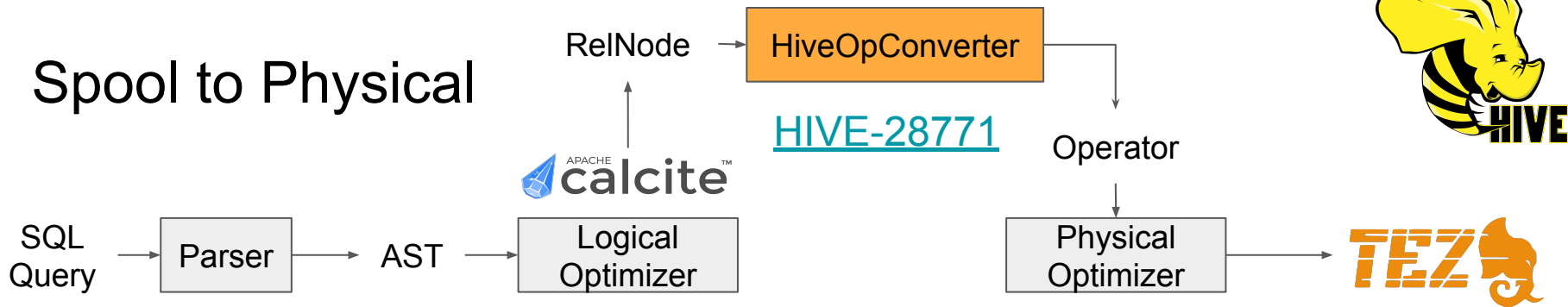
Spool to Physical



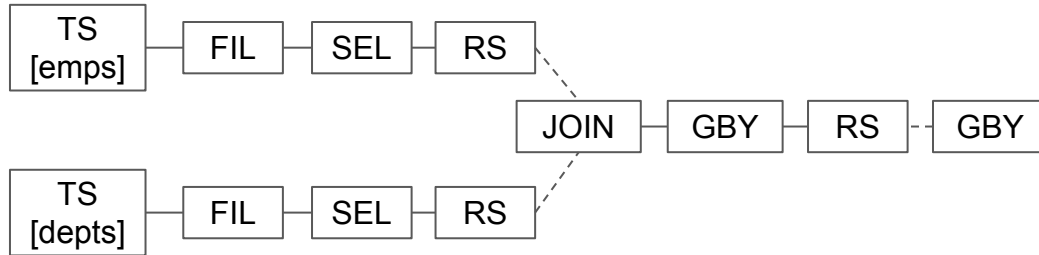
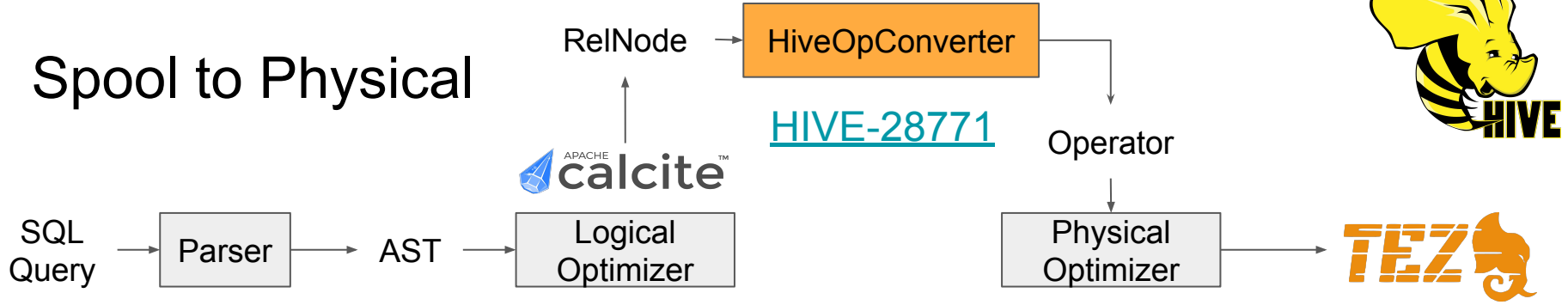
Spool to Physical



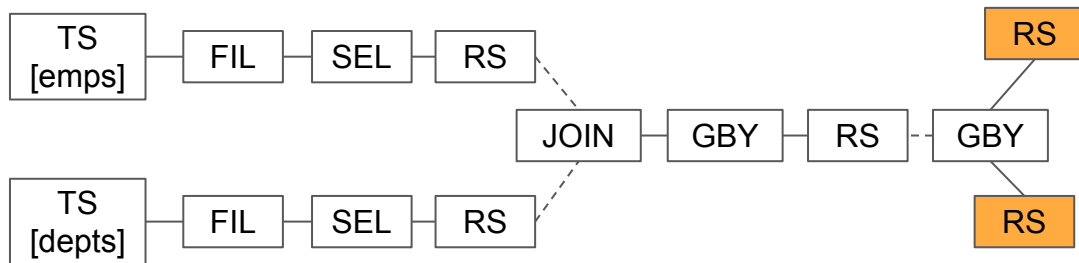
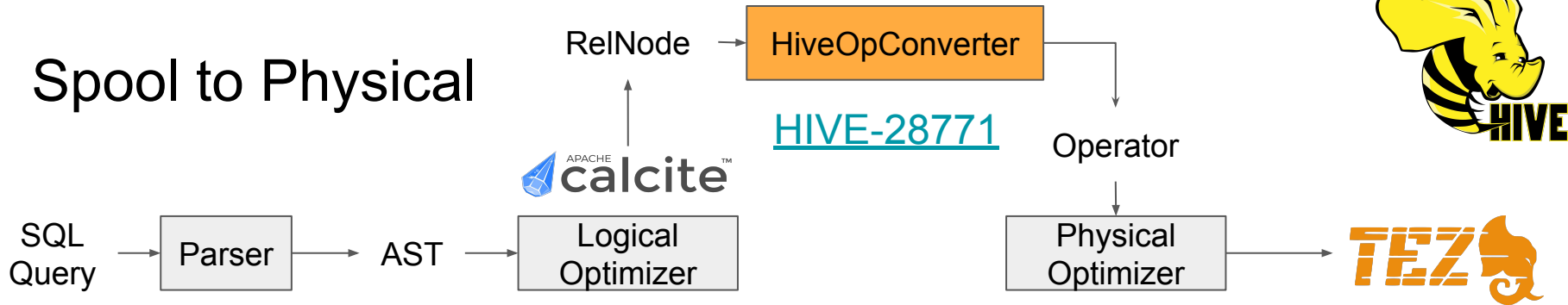
Spool to Physical



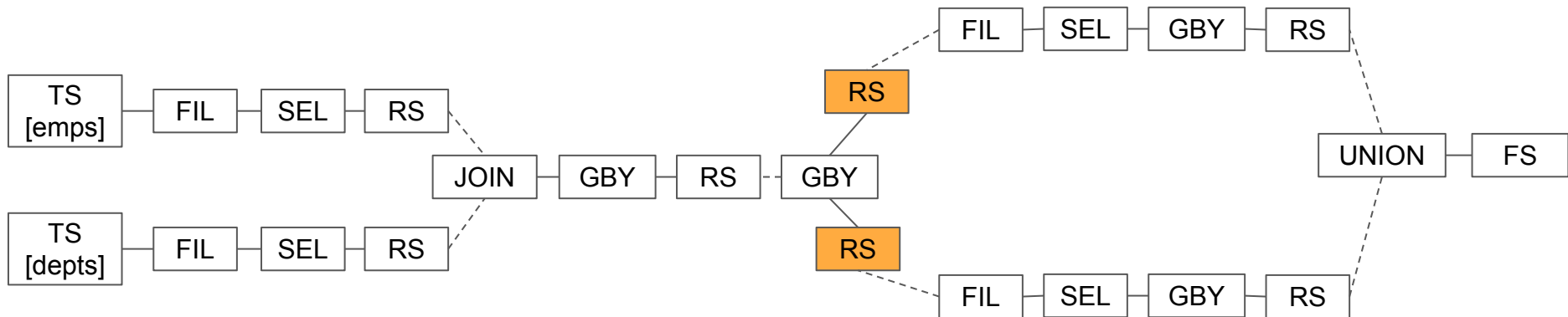
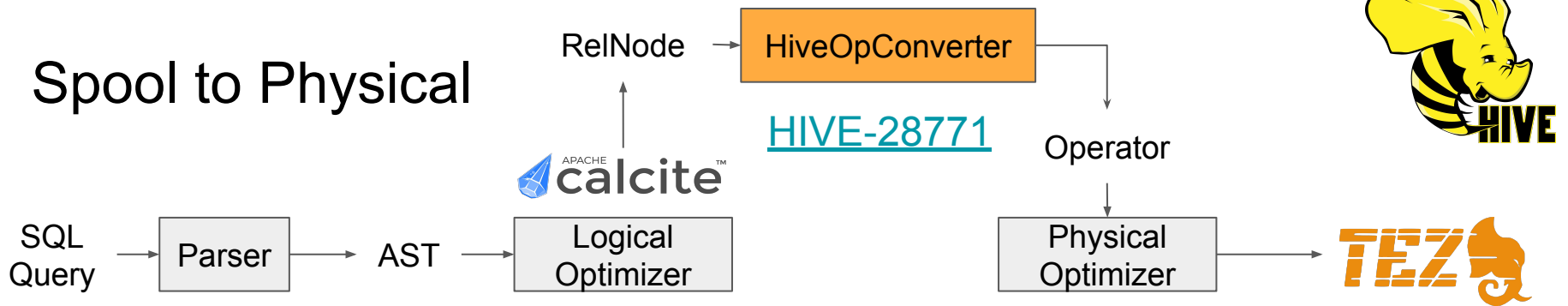
Spool to Physical



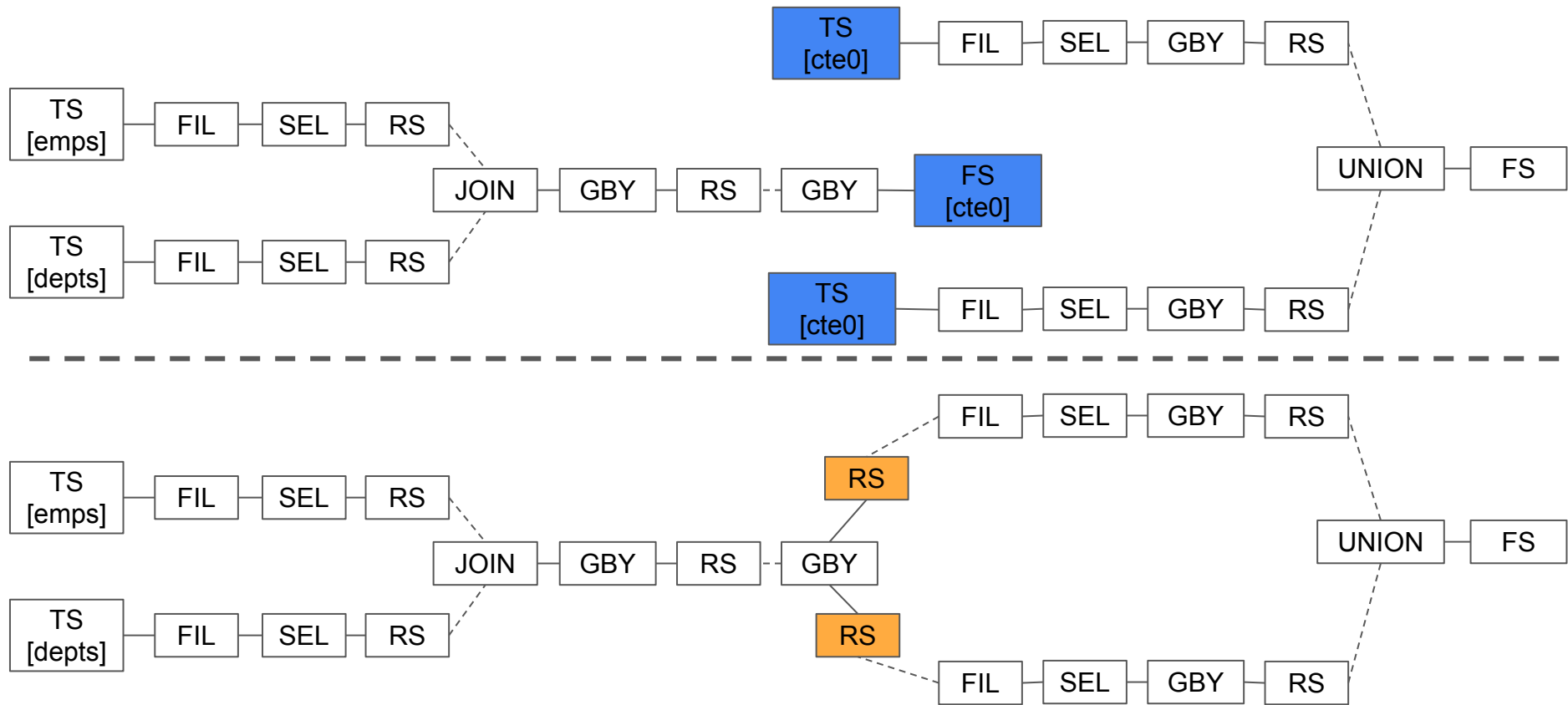
Spool to Physical



Spool to Physical



Spool to Physical



Conclusions

Common table expressions (CTEs) are **omnipresent** in SQL workloads

Query **cost** can dramatically **decrease** by exploiting CTEs

Apache Hive utilizes Apache Calcite for **optimizing** queries with **CTEs**

CTEs are modelled as **materialized views** (MV)

TableSpool/TableScan operators are used to establish **producer/consumer** relationships

Spools are transformed to a Hive **physical** plan via:

- **ASTConverter** and transformation to **WITH** clauses
- **HiveOpConverter** by connecting producers/consumers with **DAG edges**

Apache Calcite provides all the primitives for easily building a CTE aware optimizer

References

Zhou, Jingren, et al. "[Efficient exploitation of similar subexpressions for query processing.](#)" Proceedings of the 2007 ACM SIGMOD international conference on Management of data. 2007.

El-Helw, Amr, et al. "[Optimization of common table expressions in mpp database systems.](#)" Proceedings of the VLDB Endowment 8.12 (2015): 1704-1715.

[HIVE-28259](#): Common table expression detection and rewrites using CBOHandle

[HIVE-28771](#): CTEs/HiveTableSpool when hive.cbo.returnpath.hiveop is true

[HIVE-24081](#): Enable pre-materializing CTEs referenced in scalar subqueries

[HIVE-11752](#): Pre-materializing complex CTE queries

Thank you